

TOPS Technologies

Interview Questions — React Advanced (Hooks, Firebase, Redux)

Total Questions: 55

1. What is the difference between props and state in React?

Hint: Think about which one is controlled by the component itself and which comes from outside.

2. Why is component reusability important when building large React applications?

Hint: Consider how changes in one place can affect maintainability and scalability.

3. Explain what is meant by 'prop drilling' and why it can be a problem in large React apps.

Hint: Think about passing data through many layers of components.

4. What is a custom hook in React and what naming convention should you follow when creating one?

Hint: Focus on the 'use' prefix and why it's required.

5. How would you convert repetitive API call logic in multiple components into a reusable custom hook?

Hint: Consider extracting the logic into a function that uses hooks internally.

6. State one rule you must follow when using hooks in React.

Hint: Think about where hooks can be called inside your code.

7. Write a simple custom hook called `useFetchData` that fetches data from a given API endpoint and returns the result.

Hint: You'll need to use `useEffect` and `useState` inside your custom hook.

8. How do custom hooks help promote reusability across multiple components in a React project?
-

9. How can you pass parameters to a custom hook, and why might you want to do this?

Hint: Think about customizing the hook's behavior for different use cases.

10. Suppose you want your `useFetchData` hook to handle loading and error states as well as data. What would you return from the hook?

Hint: Consider returning an object with multiple properties.

11. Give an example of when you might need to use multiple `useEffect` hooks inside a custom hook.

12. What is a reducer function in React, and how does it relate to the `useReducer` hook?

Hint: Think about how reducers are used to manage state updates based on actions.

13. Write a simple reducer function and show how you would use the `useReducer` hook to implement a counter with increment, decrement, and reset actions.

Hint: You'll need an initial state and a switch/case or if/else inside the reducer.

14. When would you prefer `useReducer` over `useState` for state management in React?

Hint: Consider the complexity and structure of the state you need to manage.

15. How do you handle multiple actions in a reducer, and how do you dispatch an action from a child component?

Hint: Think about the action object and the dispatch function.

16. Describe how you would build a simple to-do app using `useReducer` for add, edit, and delete actions.

17. What is React re-rendering, and how can it cause performance bottlenecks in large lists?

Hint: Focus on what triggers a component to re-render.

18. How does the `useMemo` hook help optimize performance in React?

Hint: Think about memoizing expensive calculations.

19. What is the difference between `useMemo` and `useCallback` ?

Hint: One memoizes values, the other memoizes functions.

20. Suppose you have a component rendering a large list of items. How would you use `useMemo` to improve its performance?

21. What is the Context API in React, and why would you use it?

Hint: Think about sharing state across many components without prop drilling.

22. How do you create a context and provide its value to deeply nested components?

Hint: Mention Provider and Consumer roles.

23. Describe how you would use the Context API to implement a global dark/light mode in a React app.

24. How does combining Context with `useReducer` help manage global state in a React application?

Hint: Think about scalable state management with both context and reducer logic.

25. What is the benefit of using memoization techniques (like `useMemo`) with context providers?

Hint: Focus on preventing unnecessary re-renders.

26. Explain how you would structure a user authentication context using both context and reducer.

27. What is Firebase, and what role does it play in a React application?

Hint: Think about backend-as-a-service and what features it offers.

28. Describe the steps to set up Firebase in a new React project.

29. Why should you use environment variables to manage Firebase API keys in your React project?

Hint: Consider security and different environments.

30. What are the benefits of using a serverless backend like Firebase for front-end developers?

31. How would you implement user sign-up and login using Firebase Authentication in React?

Hint: Mention the use of Firebase Auth APIs.

32. What is the purpose of `onAuthStateChanged()` in Firebase, and how does it help maintain persistent authentication?

Hint: Think about listening for user login/logout events.

33. How would you securely handle user session state and implement a logout feature in your React app using Firebase?

34. What is Firestore, and how does its data model (collections and documents) differ from a traditional SQL database?

Hint: No tables or rows—think about documents inside collections.

35. Write the code to add a new task to a Firestore collection and fetch all tasks for display in a React component.

36. How would you update and delete a document in Firestore from a React app?

37. Explain how to implement real-time updates in your React app using Firestore's `onSnapshot()`.

Hint: Think about subscribing to changes in a collection or document.

38. How does Firebase automatically sync UI state with the database in real time?

39. What are the main parts of Redux architecture, and what is the role of each one?

Hint: Store, action, reducer, dispatch.

40. Describe how you would set up Redux in a React project and create a simple counter using a Redux store.

41. How do you connect a React component to the Redux store using `useSelector` and `useDispatch`?

Hint: Think about reading state and dispatching actions.

42. What is the difference between using Context API and Redux for managing state in large-scale React applications?

43. What is Redux Thunk, and why is it used for async operations in Redux?

Hint: Think about middleware and handling API calls.

44. Write a Redux thunk action to fetch a list of tasks from Firebase and update the Redux store.

45. What is the difference between controlled and uncontrolled components in React forms?

Hint: Focus on how the form data is managed.

46. How does Formik simplify form handling and validation compared to manual React form management?

47. Show how you would use Formik and Yup to validate a task creation form in React.

48. Why is it important to secure API keys using environment variables, and how would you do this in a React project?

Hint: Think about `.env` files and not exposing secrets in code.

49. How would you set up different environment configurations (development vs production) in a React app?

50. In your mini project 'Task Manager App', how would you structure authentication and task CRUD logic using Redux and Firebase together?

51. Describe the steps to deploy a React + Firebase app using Firebase Hosting or Netlify.

Hint: Include build, configuration, and deployment steps.

52. If your deployed app is not updating with the latest code changes, what steps would you take to debug and resolve the issue?

53. How could you use GitHub Copilot or ChatGPT to help you write reducers or CRUD components in your React project, and what should you check before using the generated code?

Hint: Focus on code accuracy, security, and project fit.

54. How would you use an AI tool like ChatGPT to help generate documentation (like a README.md) for your React + Firebase project, and what would you review before sharing it?

Hint: Think about project-specific details and accuracy.

55. When building a real-world project, how would you decide whether to use Context API, Redux, or both for state management?

Hint: Consider the size of the app and complexity of the state.
